CS 3510 Section B: Design & Analysis of Algorithms January 10th 2024

Problem Set 1: Big-$\mathcal{O}$ and Divide & Conquer

Ethan Loo Yu Teng Due: January 17th 2024

- Please type your solutions using LaTeX or any other software. Handwritten solutions will not be accepted.
- Your algorithms must be in plain English & mathematical expressions, and the pseudo-code is optional. Pseudo-code, without sufficient explanation, will receive no credit.
- Unless otherwise stated, all logarithms are to base two.
- If we ask for a specific running time, a correct solution achieving it will receive full credit even if a faster solution exists.

1.) (20 points) For the following list of functions, cluster the functions of the same order (i.e., f and g are in the same group if and only if $f = \Theta(g)$ into one group, and then rank the groups in increasing order of growth. You do not have to justify your answer.

(a.) $(\log n)^{\log n}$

(b.) n^{π}

(c.) 3^{2187}

(d.) $50n^{2^{\log 100}} + n$ (Note: $\log 100$ is an exponent to 2)

(e.) $9^{3\log_3 n}$ (Note its 9 to the power of 3 $\log_3 n$)

(f.) $2048^{\log n}$

(g.) $n^{\log\log n}$

(h.) $n\log(n^{15}) + 7n!$

(i.) $n\sqrt{n^7}$

(j.) $\log(n!)$

Solution:

(a) Constant order: 3^{2187}

(b) Logarithmic order: $(\log n)^{\log n}$

(c) Linearithmic order: $\log(n!)$

(d) Polynomial order: n^{π}, $2048^{\log n}$, $50n^{2^{\log 100}} + n$, $n\sqrt{n^7}$, $9^{3\log_3 n}$

(e) Factorial order: $n^{\log\log n}$, $n\log(n^{15}) + 7n!$

2.) (15 points) Prove or disprove (with a counterexample) the following claims:

(a.) If $f(n) = \mathcal{O}(p(n))$ and $g(n) = \mathcal{O}(q(n))$, then $f(n) - g(n) = \mathcal{O}(p(n) - q(n))$

Solution: Disprove using counterexample

Let $p(n) = n^2$, $q(n) = n$.
Let $f(n) = \mathcal{O}(n^2)$, $g(n) = \mathcal{O}(n)$.

$f(n) - g(n) = \mathcal{O}(n^2) - \mathcal{O}(n) \equiv \mathcal{O}(n^2) \neq \mathcal{O}(n^2 - n)$
Therefore, the statement $f(n) - g(n) = \mathcal{O}(p(n) - q(n))$ is false.

(b.) There are no functions $f(n)$ and $g(n)$ such that $f(n) \neq \mathcal{O}(g(n))$ and $g(n) \neq \mathcal{O}(f(n))$.

Solution: Disprove using counterexample

Let $f(n)$ be such that when n is even, $f(n) = n^2$ and when n is odd, $f(n) = n$.
Let $g(n)$ be such that when n is odd, $g(n) = n^2$ and when n is even, $g(n) = n$.

When n is even, $f(n) = n^2, g(n) = n$ and $g(n)$ is not in $\mathcal{O}(f(n)) = \mathcal{O}(n^2)$.
When n is odd, $g(n) = n^2, f(n) = n$ and $f(n)$ is not in $\mathcal{O}(g(n)) = \mathcal{O}(n^2)$.
Therefore, there are functions such that $f(n) \neq \mathcal{O}(g(n))$ and $g(n) \neq \mathcal{O}(f(n))$.

3.) (20 points) Assume n is a power of 6. Assume we are given an algorithm $f(n)$ as follows:

```
function f(n):
   if n>1:
      for i in range(34):
        f(n/6)
      for i in range(n*n):
        print("Banana")
      f(n/6)
   else:
      print("Monkey")
```

(a.) What is the running time for this function $f(n)$? Justify your answer. (Hint: Recurrences)

Solution: $a = 34 + 1, b = 6, d = 2$
$T(n) = aT(\frac{n}{b}) + \mathcal{O}(n^d)$
$T(n) = 35T(\frac{n}{6}) + \mathcal{O}(n^2)$

$\log_b a = \log_6 35 = 1.9842 < 2 = d$
$\therefore T(n) = \mathcal{O}(n^2)$

(b.) How many times will this function print "Monkey"? Please provide the exact number in terms of the input n. Justify your answer.

Solution: There are $\log_6 n$ levels in the recursion tree. Each node on the deepest level prints "Monkey". At each level i, there are $35^i = 35^{\log_6 n}$ nodes. Therefore "Monkey" will be printed $35^{\log_6 n}$ times.

4.) (20 points) Rohit and Saigautam are trying to come up with Divide & Conquer approaches to a problem with input size n. Rohit comes up with a solution that utilizes 27 subproblems, each of size $n/9$ with time $n\sqrt{n} + n \log n$ to combine the subproblems. Meanwhile, Saigautam comes up with a solution that utilizes 17 subproblems, each of size $n/4$ with time $n^2 + n \log^2 n$ to combine.

(a.) What is the runtime of both algorithms? Which one runs faster, if either?

Solution:

Rohit: $T(n) = 27T(\frac{n}{9}) + \mathcal{O}(n\sqrt{n} + \log(n))$
$\equiv 27T(\frac{n}{9}) + \mathcal{O}(n\sqrt{n})$

$a = 27, b = 9, d = 1.5$
$\log_b a = \log_9 27 = 1.5 = d$
$\therefore T(n) = n^{1.5} \log n$

Saigautam: $T(n) = 17T(\frac{n}{4}) + \mathcal{O}(n^2 + n \log^2 n)$
$\equiv 17T(\frac{n}{4}) + \mathcal{O}(n^2)$

$a = 17, b = 4, d = 2$
$\log_b a = \log_4 17 > 2 = d$
$\therefore T(n) = \mathcal{O}(n^{\log_4 17}) \approx \mathcal{O}(n^{2.0437})$

Rohit's algorithm runs faster.

(b.) Let's say Diksha also tries to solve the same problem using an algorithm of her own. She utilizes 11 sub-problems of size $n/6$ with time $\log n$ to combine subproblems. Is this algorithm faster than the one you chose in part (a)? Why or why not?

Solution:

Diksha: $T(n) = 11T(\frac{n}{6}) + \mathcal{O}(\log n) \leq 11T(\frac{n}{6}) + \mathcal{O}(n)$
$\log_b a = \log_6 11 \approx 1.3383$

$\log_b a = \log_6 11 \approx 1.3383 > 1 = d$
$\therefore T(n) = \mathcal{O}(n^{\log_6 11}) \approx \mathcal{O}(n^{1.3383})$

Diksha's algorithm has a runtime of $\mathcal{O}(n^{1.3383})$ while Rohit's algorithm has a runtime of $\mathcal{O}(n^{1.5} \log n)$. **Diksha's algorithm is faster.**

5.) (20 points) Assume that n is a power of two. The Hadamard matrix H_n is defined as follows:

$$H_1 = \begin{bmatrix} 1 \end{bmatrix}$$
$$H_2 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$
$$H_n = \begin{bmatrix} H_{n/2} & H_{n/2} \\ H_{n/2} & -H_{n/2} \end{bmatrix}$$

Design an $O(n \log n)$ algorithm that calculates the vector $H_n v$, where n is a power of 2 and v is a vector of length n. Justify the runtime of your algorithm by providing a recurrence relation and solving it. (Hint: you may assume adding two vectors of order n takes $\mathcal{O}(n)$ time.)

Solution:

```
def Hadamard(H_n, v){
    if n == 1
        return v[0]

    v_top = [:v_{n//2}]
    v_bottom = [v_{n//2}:]
    Hadamard(H_{n//2}, v_top)
    Hadamard(H_{n//2}, v_bottom)

    result_top = H_{n}v_top + H_{n}v_bottom
    result_bottom = H_{n}v_top - H_{n}v_bottom
    return result_top + result_bottom
}
```

This is an $\mathcal{O}(n \log n)$ algorithm since:
$a = 2$(branching factor), $b = 2$(size of subproblem), $d = 1$(recombination of matrices is O(n))

$\log_2 2 = 1 = d$
$\therefore T(n) = \mathcal{O}(n \log n)$